

Project Bellum: Ultimate Boredum Cure

Problem Statement:

In this past year, our group has seen countless cases of people around the world and in PSHS-CAR Campus feeling bored and horrible from having nothing to do. To fix this issue our group has come up with the ultimate cure and solution...and that is Project Bellum: The Ultimate Boredom Cure! This is an RPG game where you can explore another worldly dimension and discover new things in a virtual set up, defeat bosses, and level up. This is for all the scholars and people who have nothing to do and those who have bad general wellbeing because when they play our game they will feel better and have better general wellbeing by having their mind stimulated or at least distracted and they can have an escape from reality whenever they feel down or are bored. We have tested our game's efficiency in curing boredom already and it is very efficient at it because one time when our good friend Nash was bored and could not play volleyball in the PSHS-CAR Campus Grounds due to rain, he resorted to playing Project Bellum: Ultimate Boredom Cure and he immediately felt way happier and his boredom was 'cured' efficiently.

Project Objectives:

Our project's realistic objectives and goals is to cure boredom and promote better general wellbeing across the world and in PSHS-CAR Campus. We want to relieve the stress of people everywhere and help people by giving them things to do whenever they're bored. We can say that we have accomplished our objectives by having 5 students to approve our game and say it is effective at curing boredom and bettering your well being.

Planned Features:

1. In our project, we have a variety of features like imports for the tools used in our game, we have the json, time, random, sys, os, shutil, msvcrt, and pygame for our imports. Json Import for reading and writing game data files. Time Import for delays in our text and typing effects. Random Import for damage variations and random damage hits. Sys Import for exiting the game. Os Import for clearing your screen. Shutil Import for centering text. Msvcrt Import for detecting key presses without needing the enter button. Pygame Import for our music and sound effects, all these imports enhance the player's game experience, helping them become more immersed in the gameplay and making it truly feel like an RPG game.
2. In our project, we also have a save and load feature where the data of the player will be saved into a json file and a function will check everytime if that json file has data in it, if there is then the function will input that data into the user's data and bring them to where they were and if there is nothing within that json file then the user's data will not be replaced.
3. In our project, we have a sound system to make the game feel more realistic and less boring. We used the Pygame Import for this, helping it be better for player engagement too.
4. In our project, we have an inventory system where the user can store items, inspect the items to see its description, name, and use. If that item is a healing item they may use it to get fully or partially healed depending on what kind of healing potion it is.
5. In our project, the player doesn't need to press the enter button whenever pressing keys which help in saving time, convenience, and better gameplay overall.
6. In our project, we have text effects to print the letters one by one, having centered text, being able to skip text or dialogue, and clearing your text or screen.
7. In our project, you may talk to NPCs for quests and finishing quests give you certain rewards and experience. Whenever you talk to them at first you will be given a quest, and

after your first interaction with that NPC you will not be able to progress in the dialogue until you have finished the quest and acquired the required items.

8. In our project, there is a combat turn-based system where the player and enemy will take turns in attacking each other until one of them dies and has no more HP left. Higher attack stats lead to more damage, higher defense leads to higher damage reduction, and higher agility leads to more likely attacks.
9. In our project, we will have a main menu where the user has the choice to play, learn information about how to play the game, objectives, and sub-objectives, and the option to quit the game.

Planned Inputs and Outputs:

1. In our project, the inputs will be keyboard inputs from the user to control their character within the game and make decisions to choose their path. This is for interacting with NPCs, using the inventory, fighting enemies, saving and loading the game, choosing actions, and finally quitting the game.
2. In our project, another input from the user will be combat inputs from the user, deciding their choices during the battle if they wanna attack, open or close their inventory, or runaway from the enemy.
3. In our project, another input would be the inventory inputs where they can select items and either use the item, inspect it, or not use it at all and just select it.
4. In our project, we will have text display as our outputs where we will display the text for dialogue, locations, combat results, inventory contents, and game menus.
5. In our project, another output will be the visual effects like the font, centered text, typing animation, and screen clearing to make the game better and more realistic.
6. In our project, another output will be sound effects for background music and effects for button clicks, attacks, and dialogue typing.

Logic Plan:

Pseudo Code of our Game:

Main Menu:

Repeat forever:

Clear the screen

Show options:

1 = Play

2 = Information

3 = Quit

Wait for player input

If player chooses 1:

Start the game

If player chooses 2:

Show the information screen

If player chooses 3:

Exit the game

Game Loop:

Start the game

Play the intro dialogue

If there is no saved location:

Set starting location

Repeat forever:

Clear the screen

Show current location

Show available actions (move, pick item, talk, fight)

Wait for player input

If input is a number:

Get the selected result

If it is a location:

Move the player there

If it is an item:

Add item to inventory

Remove item from the map

If it is an NPC:

Start dialogue with NPC

If it is an enemy:

Start combat

If player presses I:

Open inventory

If player presses H:

Show help/information

If player presses S:

Save the game

If player presses L:

Load saved game

If player presses X:

Ask if player wants to quit

Inventory System:

When inventory is opened:

Show all item slots

Repeat:

Wait for input

If player presses SPACE:

Close inventory

If player presses a number:

Select that item

If item is not empty:

Show item details

Item Usage System:

When player uses an item:

If the item is for healing:

If it is a full heal:

Set HP to max

Otherwise:

Add healing amount to HP

Make sure HP does not exceed max

Remove the item from inventory

Dialogue System:

When talking to an NPC:

Check each dialogue stage

If conditions are met (like having required items):

Show dialogue

Remove required items

Give reward if needed

Update quest progress

Stop checking

If no conditions are met:

Show “incomplete” dialogue

Combat System:

When combat starts:

Reset enemy HP

Repeat while both player and enemy are alive:

Show player and enemy stats

Player's turn:

Choose:

1 = Attack

2 = Inventory

3 = Run

If attack:

Check if attack hits

If yes:

Deal damage

If no:

Show miss message

If inventory:

Open inventory

If run:

Exit combat

If enemy is still alive:

Enemy's turn:

Check if attack hits

If yes:

Deal damage to player

If no:

Show miss message

Wait for player to continue

After combat:

If player HP is 0:

Show defeat message

If enemy HP is 0:

Show victory message

Give loot

Save and Load System:

When saving:

Store player stats, inventory, and location into a file

When loading:

Read data from file

Restore player stats, inventory, and location